Course ID: SW 302
User Interface (UI) Development

Lecture#9
Introduction React

Lecture Outline

React

Fetch API for API Calls

What is fetch()?

- Modern way to make **HTTP requests** in the browser
- Returns a **Promise**
- Typically used with **JSON APIs**

GET Request

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error:", error));
```

`fetch()` is **Promise-based** → asynchronous

Use `.then()` or `async/await` for handling responses

`.then()` → executes when the Promise **resolves**

`.catch()` → executes when the Promise **rejects**

POST Request

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hello", body: "World" })
})
.then(res => res.json())
.then(data => console.log("Created:", data))
.catch(err => console.error(err));
```

JS continues running the next lines **without waiting**

When the Promise **resolves**, the `.then()` callback runs

JSON

What is JSON?

JSON (JavaScript Object Notation). A **lightweight text format** for storing and exchanging data. Based on JavaScript object syntax, but used by **all programming languages**.

Commonly used for:

- APIs, Configuration files , Data interchange between frontend & backend

Rules

Data is written as key–value pairs

Keys must be in double quotes

Strings must use double quotes

Allowed data types:
`string`, `number`, `boolean`, `null`
`object` (curly braces `{}` → key-value pairs)
`array` (square brackets `[]` → ordered list)

Example

```json
{
  "name": "Maria",
  "age": 22,
  "isStudent": true,
  "skills": ["HTML", "CSS", "JavaScript"],
  "address": {
    "city": "Giza",
    "zip": 12345
  }
}
```

Working With JSON in JavaScript

1. Converting JSON → JavaScript Object

JSON comes as a string, usually from a server or API.
Use `JSON.parse()` to turn it into a JS object.

```
let jsonString = '{"name": "Ali", "age": 20, "skills": ["HTML","CSS"]}';
let user = JSON.parse(jsonString);

console.log(user.name); // "Ali"
console.log(user.skills[1]); // "CSS"
```

Turns a JSON string into a usable JS object

Required to read API responses

2. Converting JavaScript Object → JSON

Use `JSON.stringify()` to convert a JS object back into a JSON string.

```
let student = {
  name: "Sara",
  grade: "A",
  active: true
};

let jsonData = JSON.stringify(student);
console.log(jsonData);
// {"name":"Sara","grade":"A","active":true}
```

Used when sending data from the browser to a server

Converts objects, arrays, booleans, numbers, null

Real Example: Fetch JSON from an API

```
fetch("https://example.com/data.json")
  .then(res => res.json())    // auto JSON.parse()
  .then(data => console.log(data));
```

Most APIs return JSON

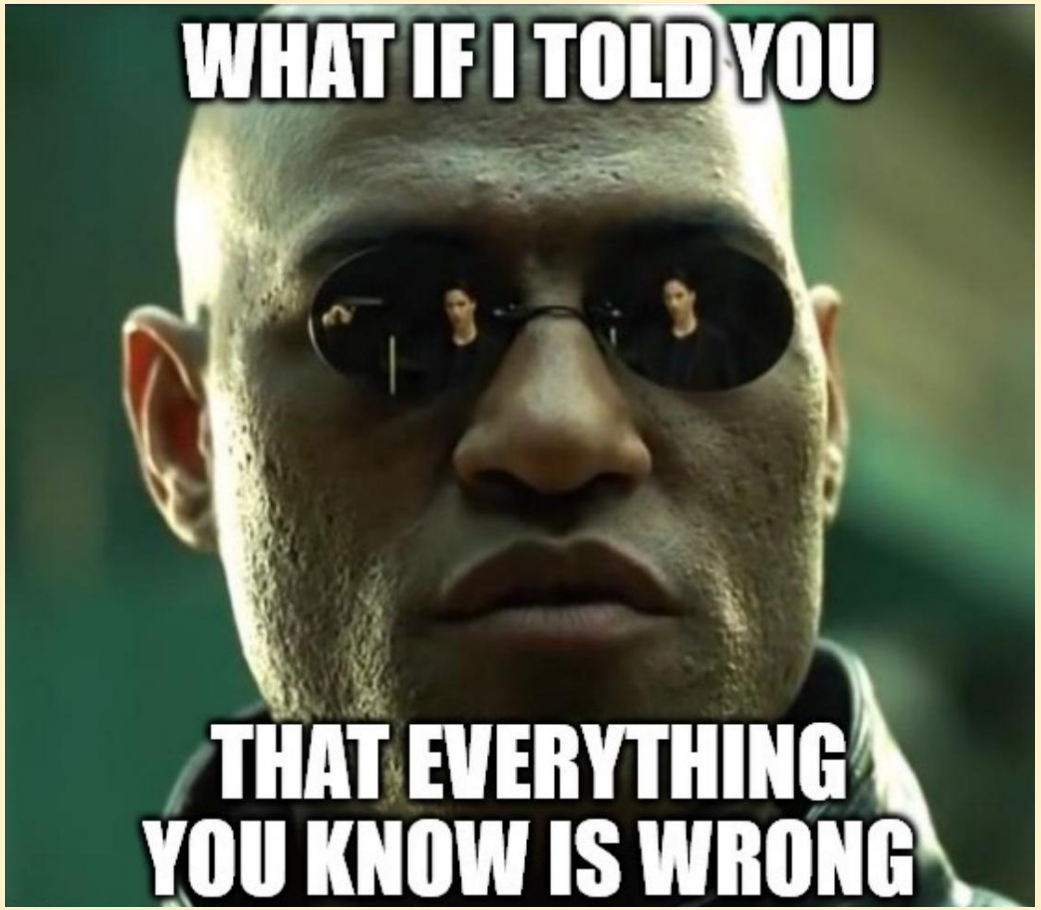
`res.json()` automatically parses it

`JSON.parse` and `JSON.stringify` Cannot Handle

- ❌ Functions
- ❌ `undefined`
- ❌ `Infinity`, `NaN`

```
JSON.stringify({ x: undefined });
// "{}"
```

React


WHAT IF I TOLD YOU
THAT EVERYTHING
YOU KNOW IS WRONG

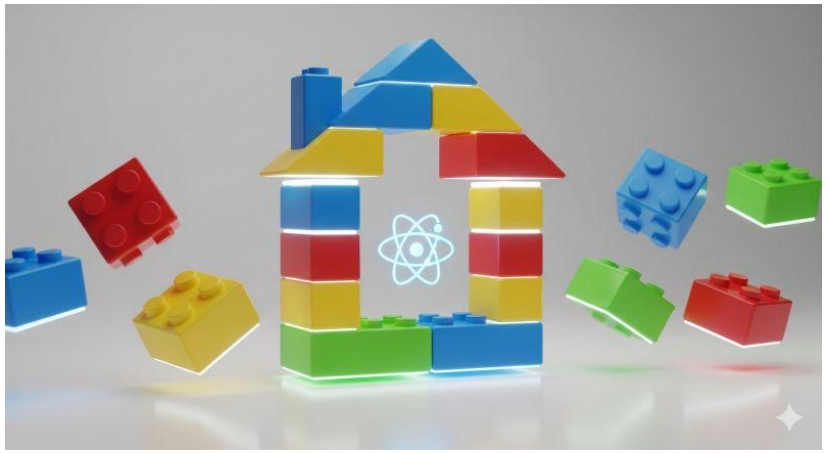
What is React?

- React is a **JavaScript library, not a strict framework** for building user interfaces (UIs), maintained by **Facebook (Meta)**

Why React?

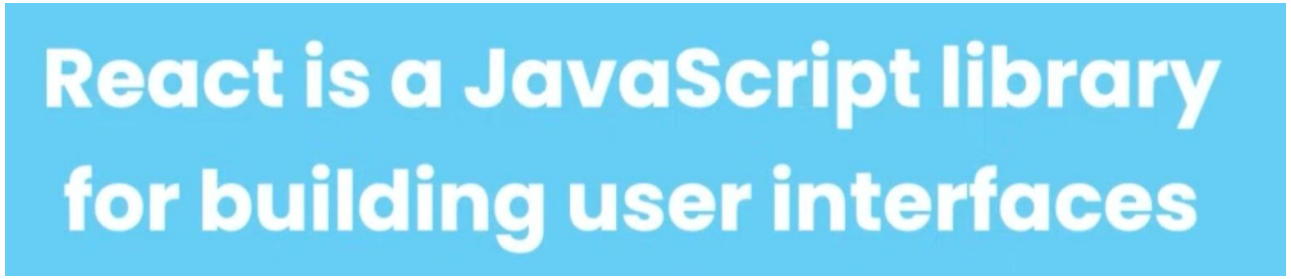
- UI automatically updates when data changes (**reactive UI**)
- Uses **components** to split UI into reusable pieces
- Slow DOM… while React offers a virtual DOM, which is faster
- Declarative syntax via **JSX** (JavaScript + HTML)

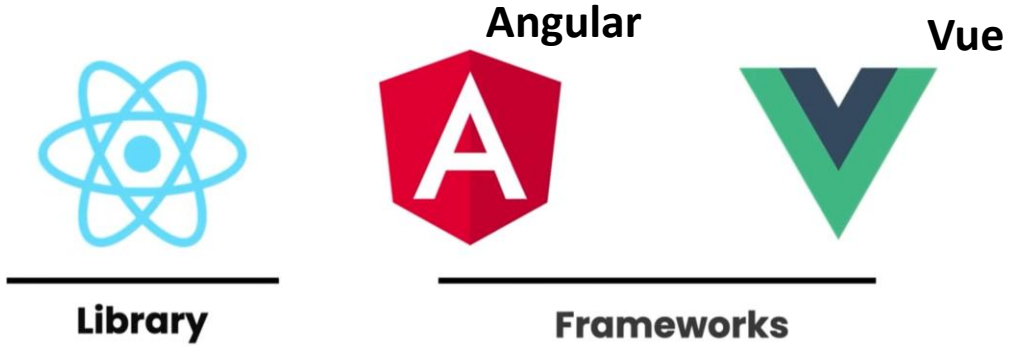
React itself focuses specifically on the "view" layer (the UI) and doesn't prescribe how to structure the rest of the app, unlike a traditional framework that dictates the entire application's architecture

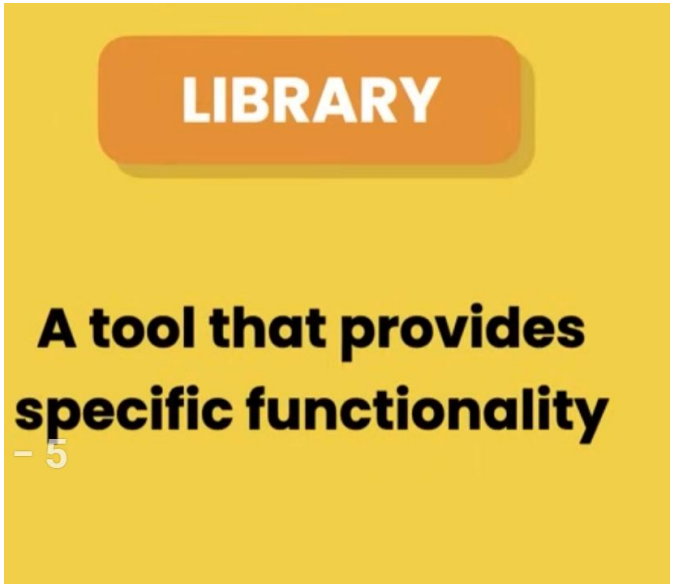


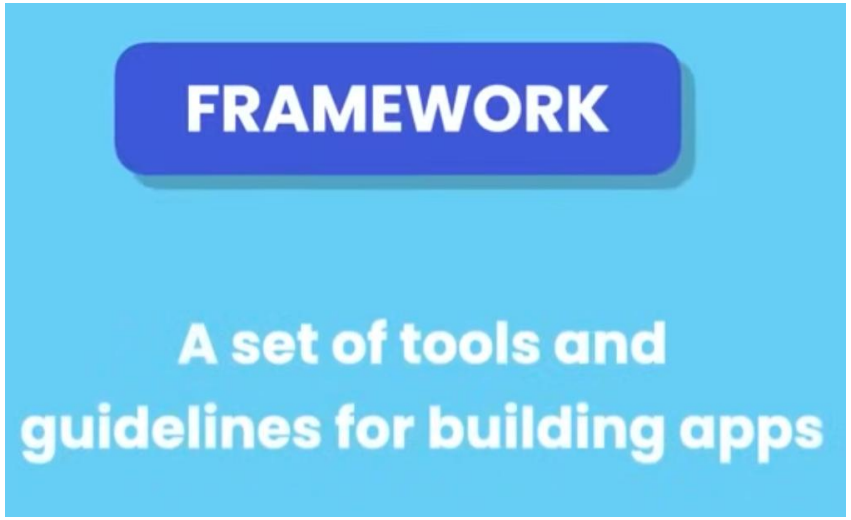
What is React?

- React is a **JavaScript library, not a strict framework** for building user interfaces (UIs), maintained by **Facebook (Meta)**









Quick Comparison Table

Feature	**React**	**Angular**	**Vue**
Type	Library	Full Framework	Progressive Framework
Developer	Meta	Google	Evan You
Language	JS + JSX	TypeScript	JS/TS + Templates
Learning Curve	Medium	High	Low
Opinionated?	❌ No	✔ Yes	⚠ Somewhat
Built-in Router	❌ No	✔ Yes	❌ (Vue Router external)
Built-in State	❌ No (Redux, Zustand, etc.)	✔ NgRx (recommended)	⚠ Pinia (official but external)
Best For	Flexible UIs	Enterprise apps	Simpler projects

Why TypeScript Exists

JavaScript is flexible but has downsides for large applications:

- No types → errors happen at runtime
- Harder to maintain large codebases
- Bugs like `"5" + 5` producing `"55"`
- Easy to make mistakes without noticing

TypeScript solves these by adding **static types**.

- **TypeScript is not a different language — it is an enhanced version of JavaScript.**

  The browser **cannot run TypeScript directly**.

  TypeScript **must be compiled into JavaScript** before running.

What Is the React Ecosystem?

React Ecosystem

- **React Core**
  - react
  - react-dom
- **Build / Tooling**
  - Vite
  - Webpack
  - Parcel
  - Create React App (older)
  - Babel
  - ESLint / Prettier
- **Routing**
  - React Router
  - Next.js (includes routing)
  - Expo Router (React Native)
- **State Management**
  - Redux
  - Redux Toolkit
  - Zustand
  - Recoil
  - Jotai
- **Data Fetching / Server State**
  - Fetch API (built-in)
  - Axios
  - React Query (TanStack Query)
  - SWR
- **UI Component Libraries / Styling**
  - Material UI (MUI)
  - Chakra UI
  - Ant Design
  - Tailwind CSS
  - Bootstrap for React
  - Styled Components
  - Emotion
- **Forms & Validation**
  - React Hook Form
  - Formik
  - Yup (validation)
  - Zod
- **Testing**
  - Jest
  - React Testing Library

- React alone is library for building interactive and dynamic UIs , alone that what it is best at!
- For other functionalities, there are other libraries to add from the React Ecosystem.

What Is the React Ecosystem?

React itself is only the UI library, but to build full, real-world applications, developers use a collection of tools and libraries around React. See the lists

- **Full Frameworks (Built on React)**
- Next.js
- Gatsby
- Remix
- Expo (React Native)
- **State/Data Architecture (advanced)**
- GraphQL
- Apollo Client
- Relay Modern

Creating a new React APP

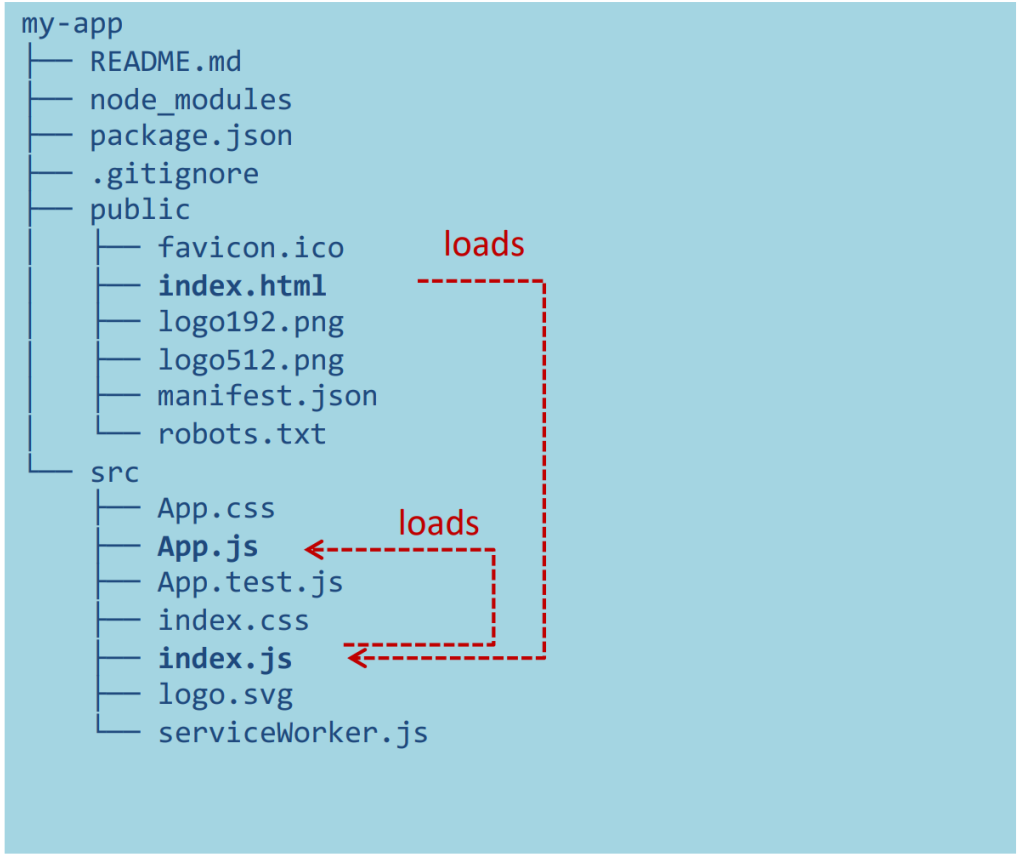
Creating a new React app is simple!

1. Install Node.js
2. Run: `npx create-react-app app-name`
3. New app created in folder: `./app-name`

Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code outside of a web browser.

Structure your React Application

Folder Structure



- `public` is the web server root
  - Static files go here
- `public/index.html` is the page template
  - Published at http://localhost:3000
  - Automatically reloads when app changes
  - No need to modify, normally
- `src` contains all scripts
- `src/index.js` is the JavaScript entry point
  - Contains the `ReactDOM.render` call to mount the App in the `#root` element
  - Do not touch, normally
- `src/App.js` is the file containing your application
  - **Develop here!**
  - Feel free to `import` other components

JSX (JavaScript + XML)

What is JSX?

HTML-like syntax inside JavaScript. JSX lets you **write UI as if you are writing HTML**, but with JavaScript power. Makes code **readable and declarative**

Combines **markup and logic in one place.**

No need for manual DOM manipulation. React optimizes updates for performance

```
const name = "Ali";
const element = <h1>Hello, {name}!</h1>;
```

```
const element = (
  <div>
    <h1>Hello</h1>
    <p>Welcome to React</p>
  </div>
);
```

- Curly braces {} are used to inject dynamic values
- JSX compiles to plain **JS under the hood**

React updates the DOM **automatically**.

JSX (JavaScript + XML)

Imperative (Plain JS, also called Vanilla JS)

```
const button = document.getElementById("btn");
button.addEventListener("click", () => {
  button.style.backgroundColor = "blue";
  button.textContent = "Clicked!";
});
```

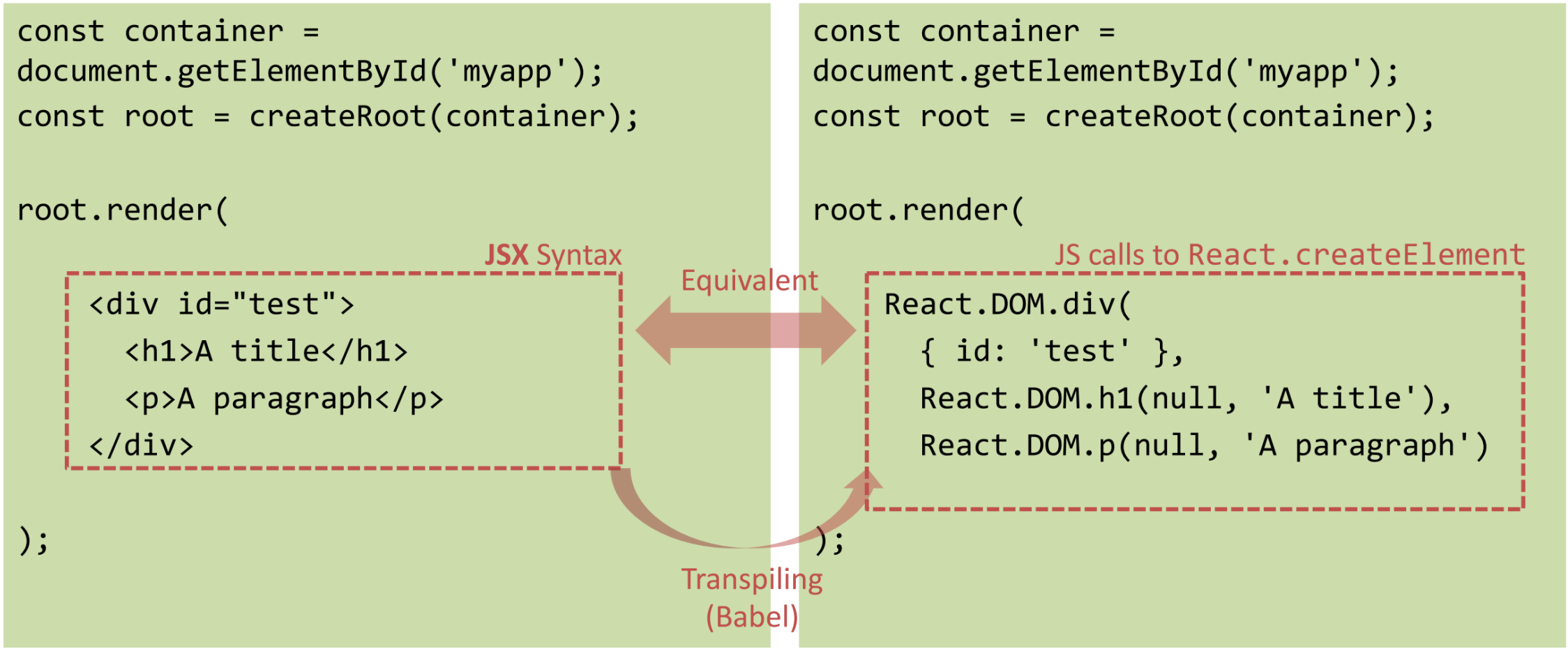
You manually tell the browser what to change.

Declarative (React JSX)

```
function App() {
  const [clicked, setClicked] = useState(false);

  return (
    <button
      onClick={() => setClicked(true)}
      style={{ backgroundColor: clicked ? "blue" : "" }}
    >
      {clicked ? "Clicked!" : "Click me"}
    </button>
  );
}
```

You only describe **the UI based on state**.
React updates everything automatically.

JSX Syntax


const container = document.getElementById('myapp');
const root = createRoot(container);
root.render(
JSX Syntax

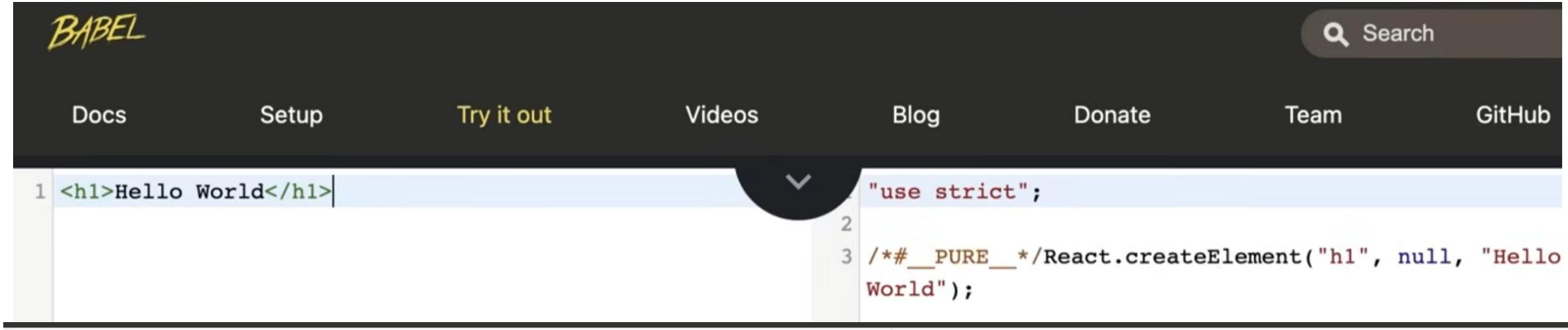
<h1>A title</h1>
A paragraph

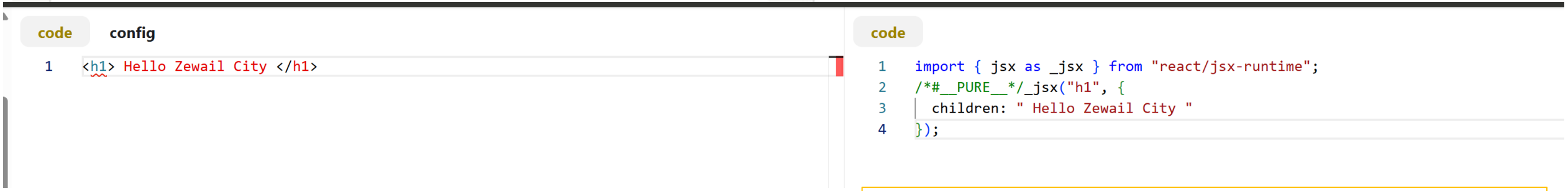
);
Equivalent
Transpiling (Babel)
const container = document.getElementById('myapp');
const root = createRoot(container);
root.render(
JS calls to React.createElement
React.DOM.div(
{ id: 'test' },
React.DOM.h1(null, 'A title'),
React.DOM.p(null, 'A paragraph')
);

JSX (JavaScript + XML)

https://babeljs.io/

JSX automatically converted to JavaScript code





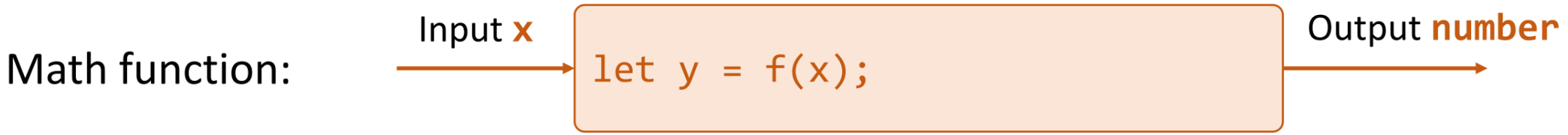
Babel is a **JavaScript compiler** that allows you to use **modern JavaScript features.** Required for **React JSX → JavaScript** conversion.

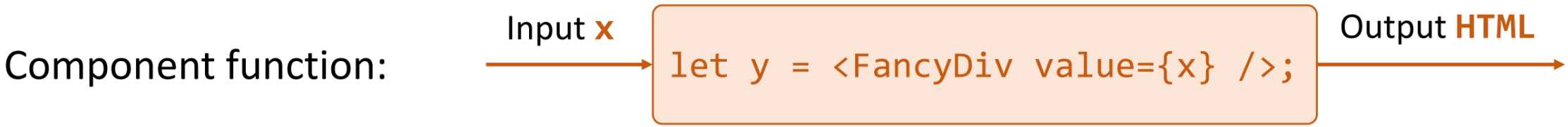
Why React Needs Babel?
React uses **JSX**, which browsers **do not understand**.

It runs automatically through your **build tool**.

React Components

Components are functions for user interfaces

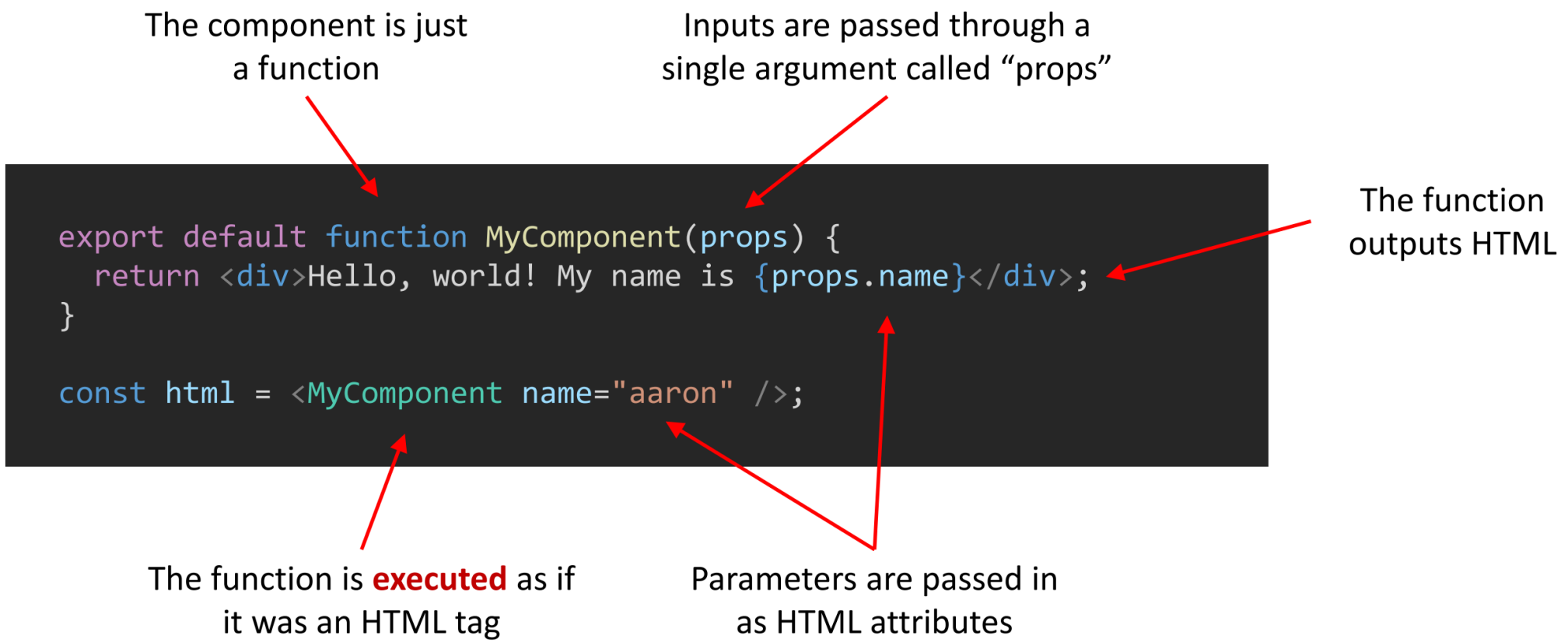




"In React, everything is a component"

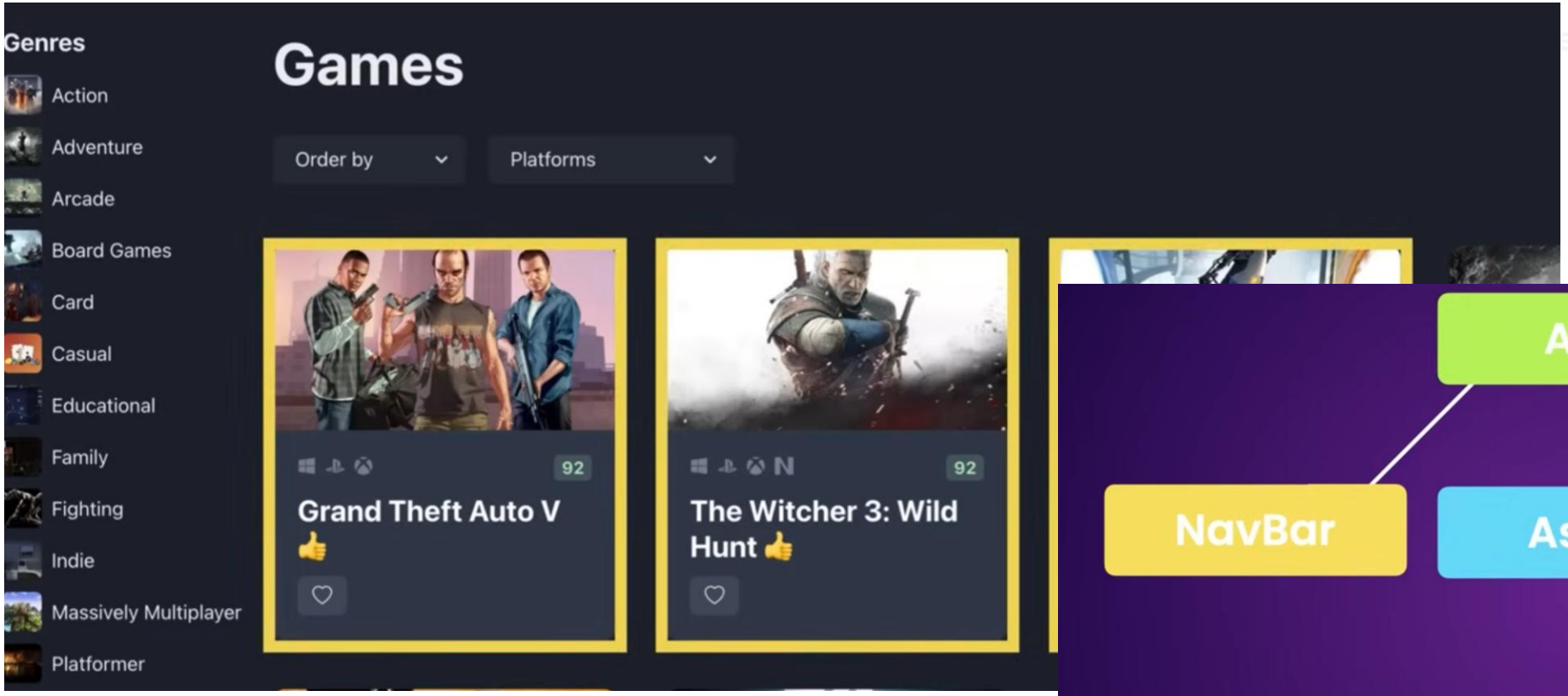
Can be a Function or a Class Component, but mostly a Function Component, as it is easy for re-use

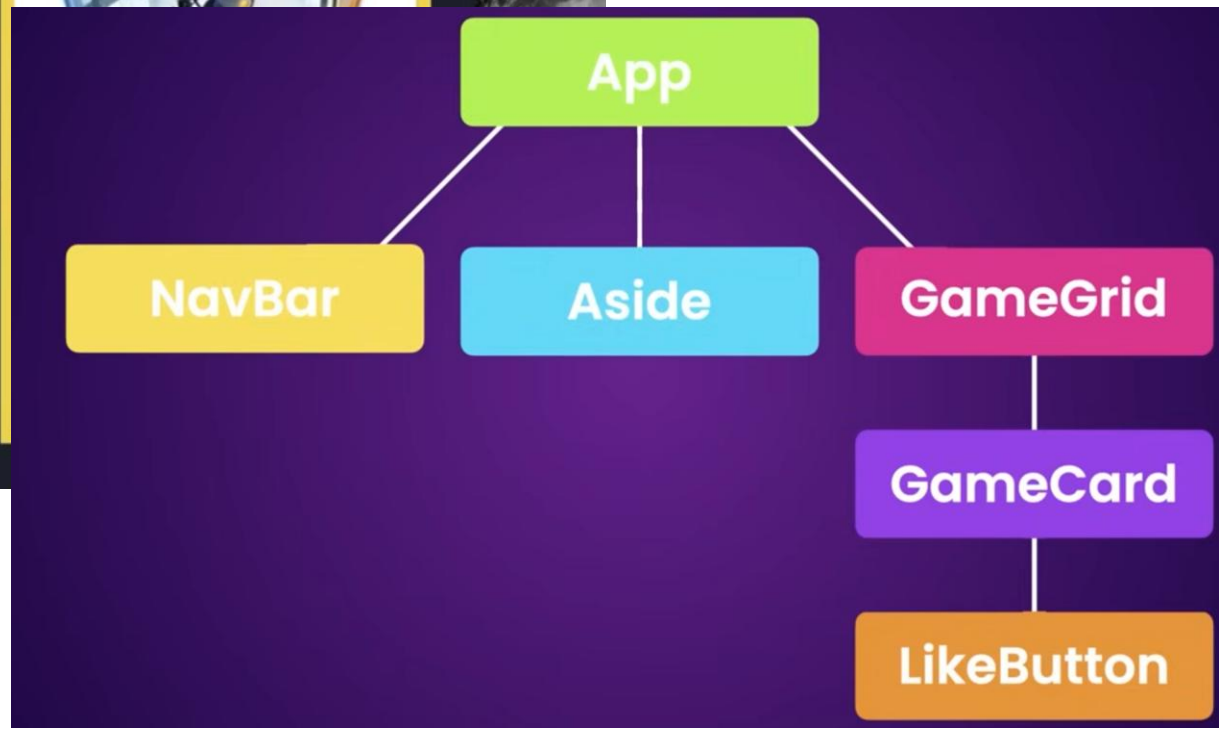
Anatomy of a React component



- When a component function **executes**, we say it “**renders**”
- Assume components may re-render at any time

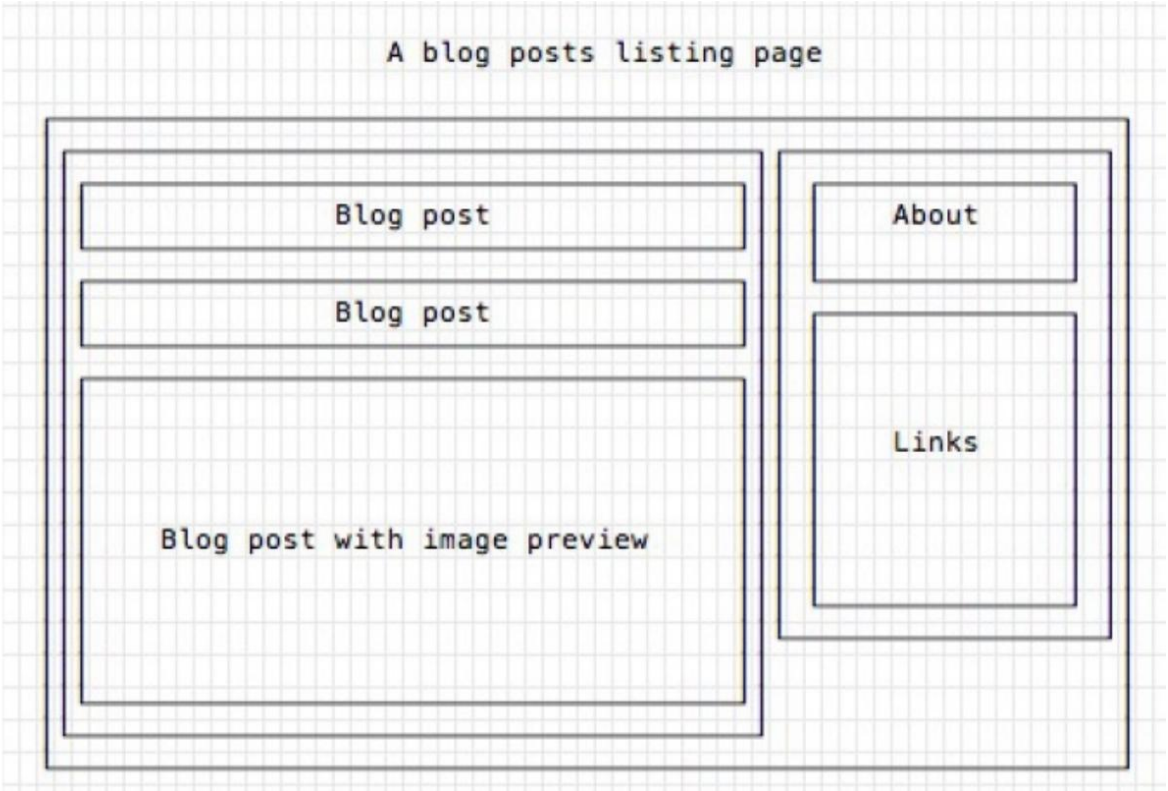
React Application is a Tree of Components where App is Root





Components

- Everything on a page is a Component
  - Even simple HTML tags (React.DOM.element)
- Components may be **nested**
- ReactDOM.createRoot().render() builds a component and attaches it to a DOM container



Basic React Component

Message Component

Message.tsx U

src > Message.tsx > default

```tsx
// PascalCasing
function Message() {
  // JSX: JavaScript XML
  return <h1>Hello World</h1>;
}

export default Message;
```

App Component

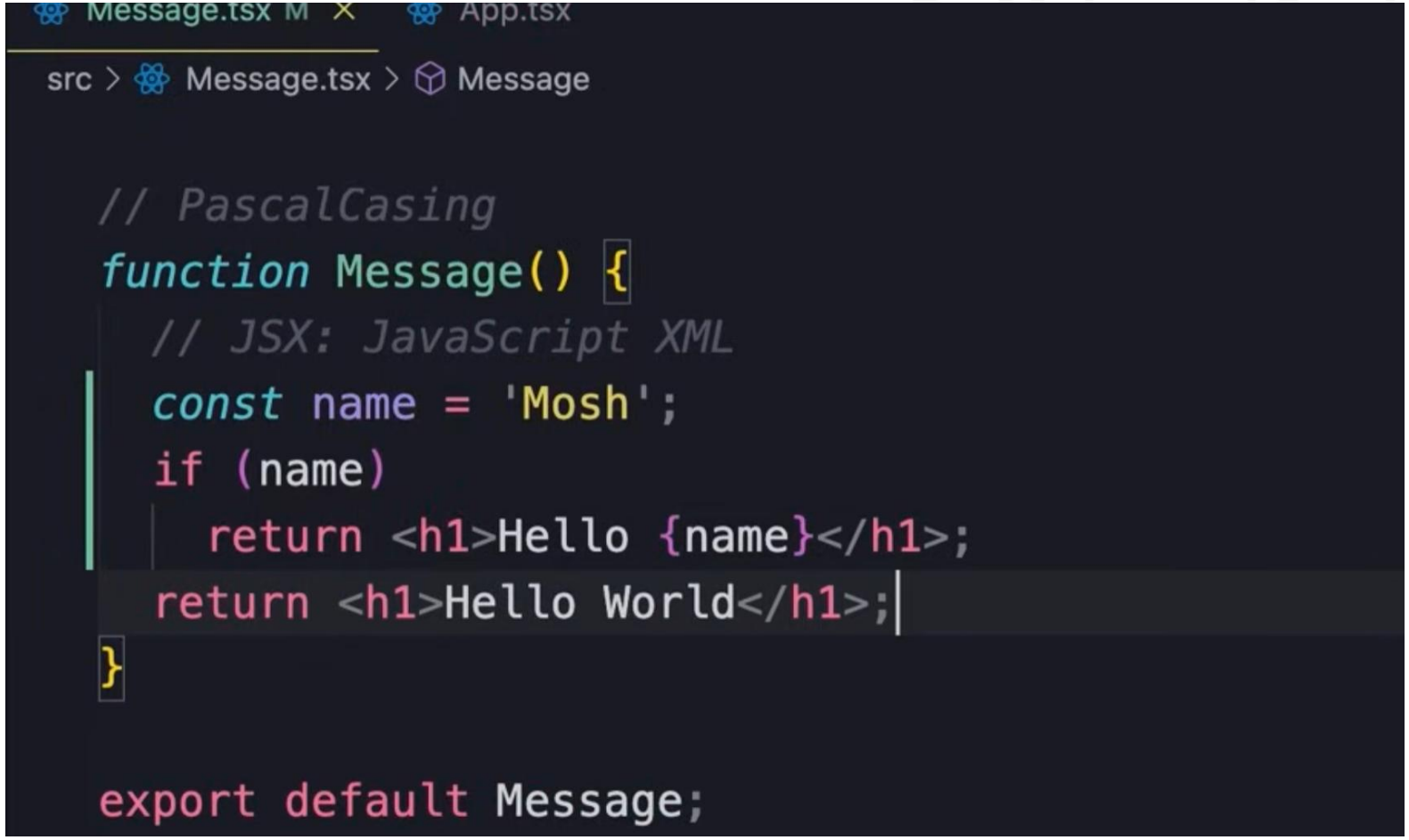
src > App.tsx > default

```tsx
import Message from './Message';

function App() {
  return <div><Message /></div>;
}

export default App;
```

In a real application, a component can have behavior, like a button. When the button is clicked, the message is changed. Etc.!

Dynamic Content: Write a JavaScript expression to produce a Value

Dynamic Content: if Condition



Types of Components

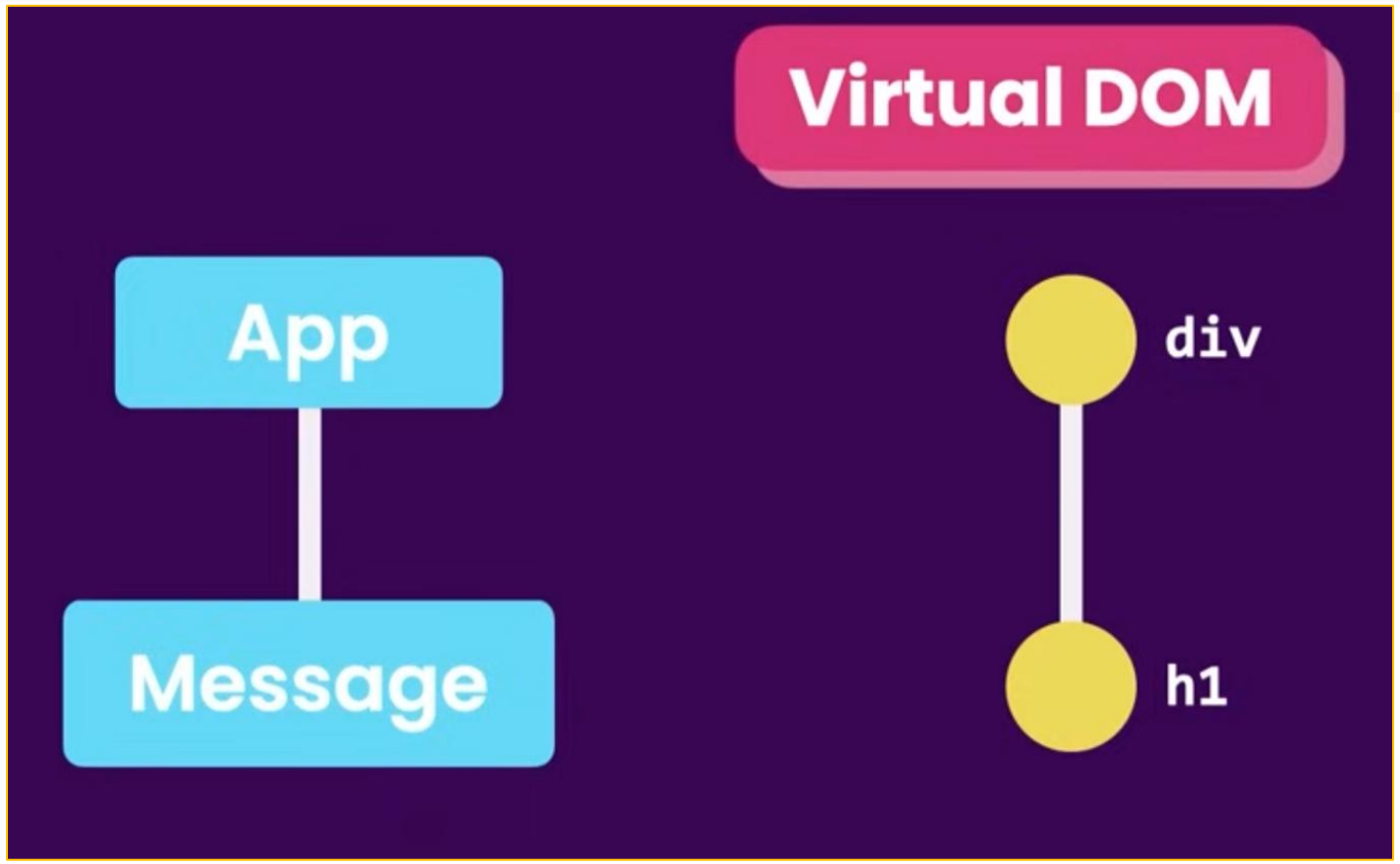
Presentational Components

- Generate DOM nodes to be displayed
- Do not manage application state
- Might have some internal state, uniquely for presentation purposes

Container Components

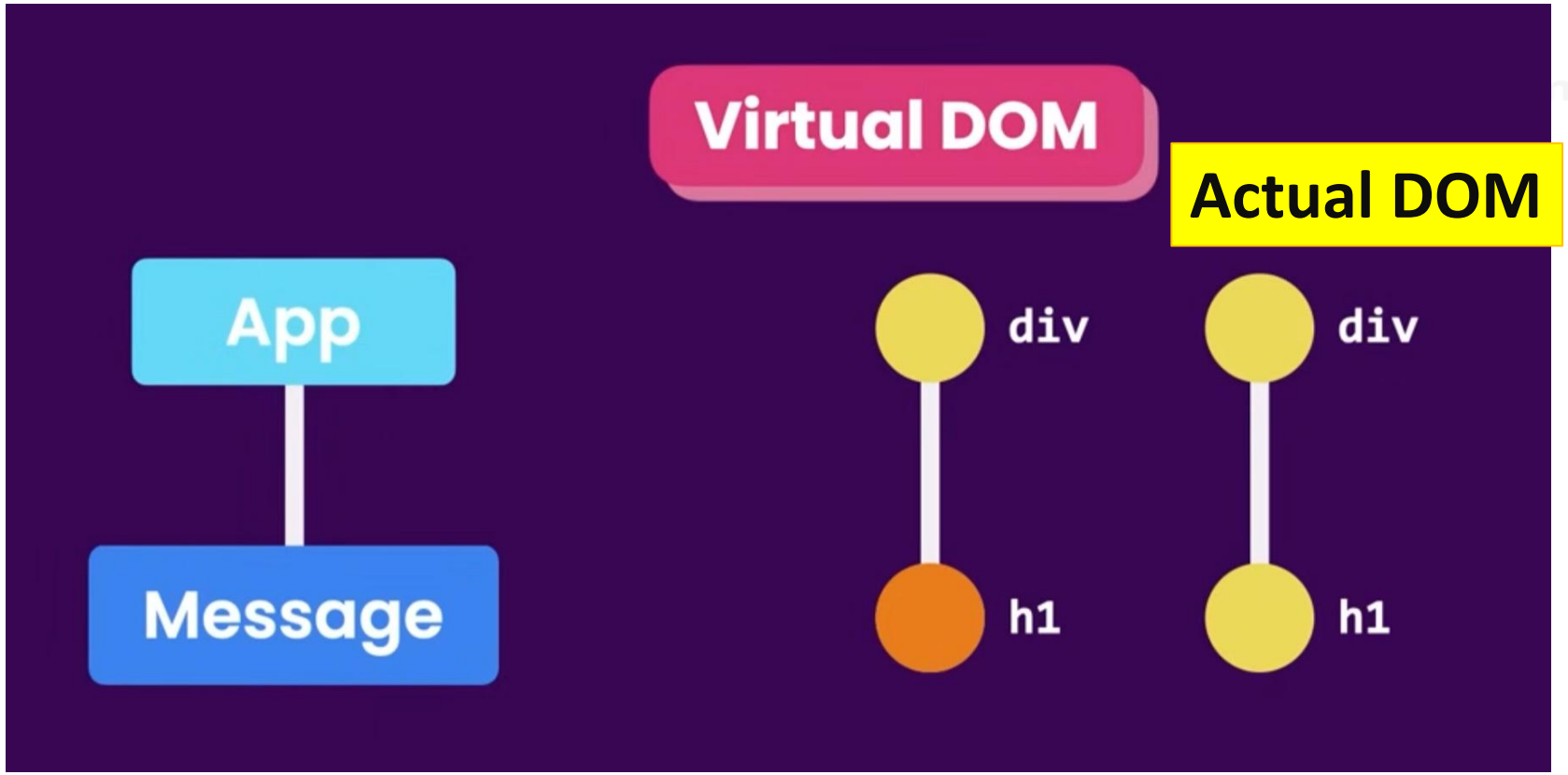
- Manage the state for a group of children
- May interact with the back-end
- Create (presentational) children to display the information

Virtual DOM



Virtual DOM is different from the actual DOM in the browser!, it is lightweight memory representation of the components tree where each node represents a component nd its properities.

Updating Virtual DOM



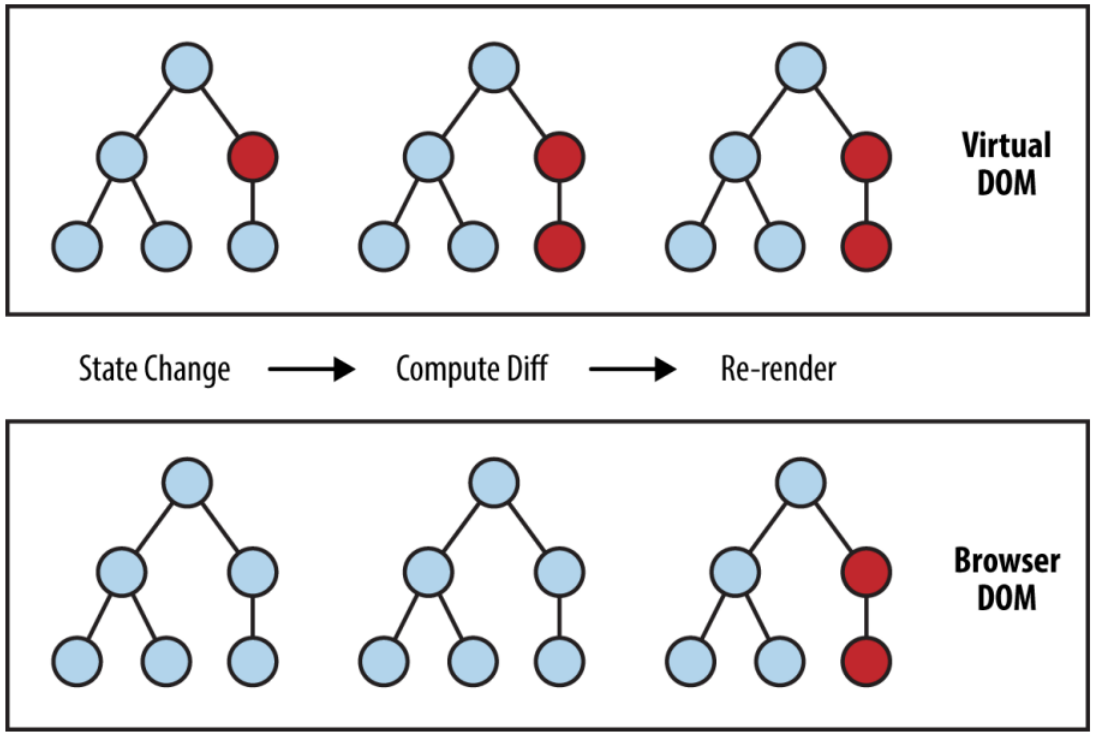
When the data of a component changes, React-DOM identifies the changed node in the Virtual DOM and compares it to the Actual DOM. Then, only update the changed corresponding nodes in the Actual DOM. The update is done by a Library called "React-DOM".

Re-Rendering Performance

- Modifications to the DOM are expensive (re-computing layout and updating GUI)
- React implements a **Virtual DOM** layer
  - Internal in-memory data structure, optimized and very fast to update
  - Corrects some DOM anomalies and asymmetries
  - Manages its own set of “synthetic” events
  - After components re-render, React computes the difference between the “old” DOM and the new modified Virtual DOM
  - Only modifications and differences are selectively applied to the browser’s DOM, in batch

Update Cycle

- Build new Virtual DOM tree
- Diff with old one
- Compute minimal set of changes
- Put them in a queue
- Batch render all changes to browser



Synthetic Events

- React implements its own event system
- A single native event handler at root of each component
- Normalizes events across browsers
- Decouples events from DOM

Updating Virtual DOM

index.html main.tsx

src > main.tsx

```tsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './index.css'

ReactDOM.createRoot(document.getElementById('root') as HTMLElement
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Main.tsx or index.tsx Is the start point of the React Application

"React-DOM" is a built-in component in React. In the Mobile Application, it has been changed to React Native. React is platform-agnostic.

Creating another Component with Style

Update App.tsx to use the new Component

ListGroup.tsx U App.tsx M

c > App.tsx > App

```tsx
import ListGroup from "./components/ListGroup";

function App() {
  return <div><ListGroup /></div>;
}

export default App;
```

For styling, we use “className” instead of “class”, Why?

ListGroup.tsx

src > components > ListGroup.tsx > ListGroup

```tsx
function ListGroup() {
  return (
    <ul className="list-group">
      <li className="list-group-item">An item</li>
      <li className="list-group-item">A second item</li>
      <li className="list-group-item">A third item</li>
      <li className="list-group-item">A fourth item</li>
      <li className="list-group-item">And a fifth one</li>
    </ul>
  );
}
```

localhost:5173

- An item
- A second item
- A third item
- A fourth item
- And a fifth one

React Fragments

In React, it is not allowed to return multiple main elements

Wrapping is a solution

ListGroup.tsx 1, M

src > components > ListGroup.tsx > ListGroup

```
function ListGroup() {
  return (
    <h1>List</h1> // React.createElement('h1')
    <ul className="list-group">
      <li className="list-group-item">An item</li>
      <li className="list-group-item">A second item</li>
      <li className="list-group-item">A third item</li>
      <li className="list-group-item">A fourth item</li>
      <li className="list-group-item">And a fifth one</li>
    </ul>
  );
```

ListGroup.tsx M

src > components > ListGroup.tsx > ListGroup

```
function ListGroup() {
  return (
    <div>
      <h1>List</h1>
      <ul className="list-group">
        <li className="list-group-item">An item</li>
        <li className="list-group-item">A second item</li>
        <li className="list-group-item">A third item</li>
        <li className="list-group-item">A fourth item</li>
        <li className="list-group-item">And a fifth one</li>
      </ul>
    </div>
  );
```

Drawback: Additional unnecessary element in the DOM, which is “Div”

React Fragments

Better solution is to use Fragments

Short Cut for Fragment

ListGroup.tsx M

src > components > ListGroup.tsx > ListGroup

```jsx
import { Fragment } from "react";

function ListGroup() {
  return (
    <Fragment>
      <h1>List</h1>
      <ul className="list-group">
        <li className="list-group-item">An item</li>
        <li className="list-group-item">A second item</li>
        <li className="list-group-item">A third item</li>
        <li className="list-group-item">A fourth item</li>
        <li className="list-group-item">And a fifth one</li>
      </ul>
```

```jsx
function ListGroup() {
  return (
    <>
      <h1>List</h1>
      <ul className="list-group">
        <li className="list-group-item">An item</li>
        <li className="list-group-item">A second item</li>
        <li className="list-group-item">A third item</li>
        <li className="list-group-item">A fourth item</li>
        <li className="list-group-item">And a fifth one</li>
      </ul>
    </>
```

Dynamic Content

Created a const items inside the component and use it. Use the JavaScript mapping function ".map" to map the content of a JavaScript array.

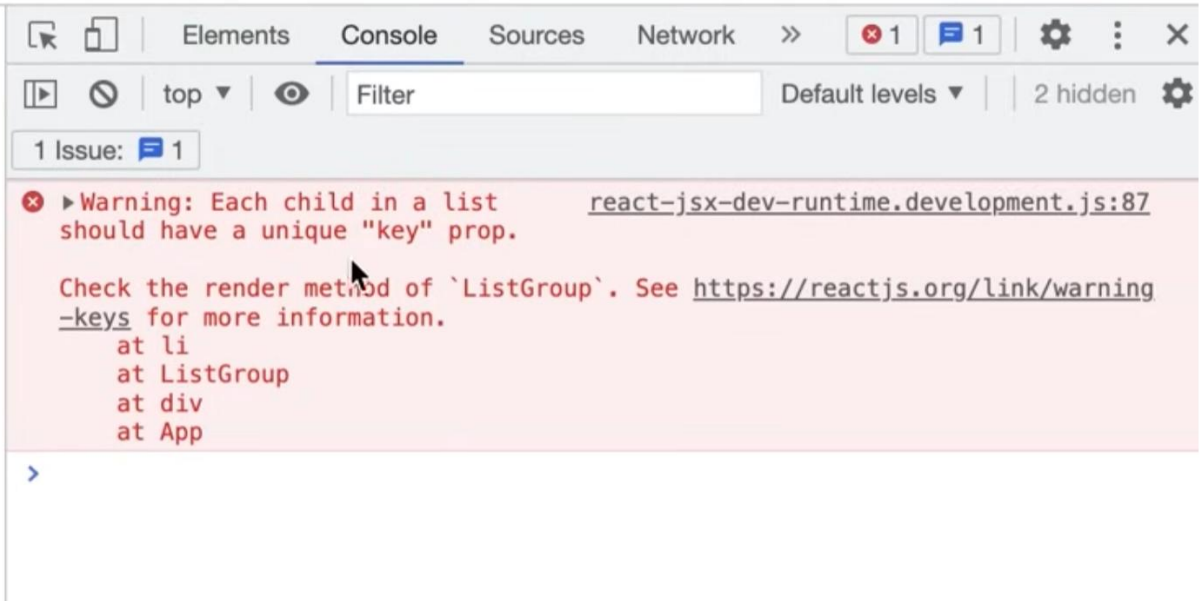
Inside the return, it is only allowed to use HTML code or React components. However, using braces "{ }" allows us to render anything dynamically.

ListGroup.tsx M

src > components > ListGroup.tsx > ListGroup > items.map() callback

```
const items = ["New York", "San Francisco", "Tokyo", "London",

return (
  <>
    <h1>List</h1>
    <ul className="list-group">
      {items.map((item) => (
        <li>{item}</li>
      ))}
    </ul>
  </>
);
}
```

It runs but gives a warning, no unique Key



Special list key property

- **Situation:** Display a **dynamic array of elements**
- Must specify a special “**key**” property for each element
- The key of an item **uniquely identifies it**
- Used by React internally for **render optimization**
- Can be any unique value (string or number)

Using Key Property

Created a const items inside the component and use it. Use the JavaScript mapping function ".map" to map the content of a JavaScript array.

ListGroup.tsx M

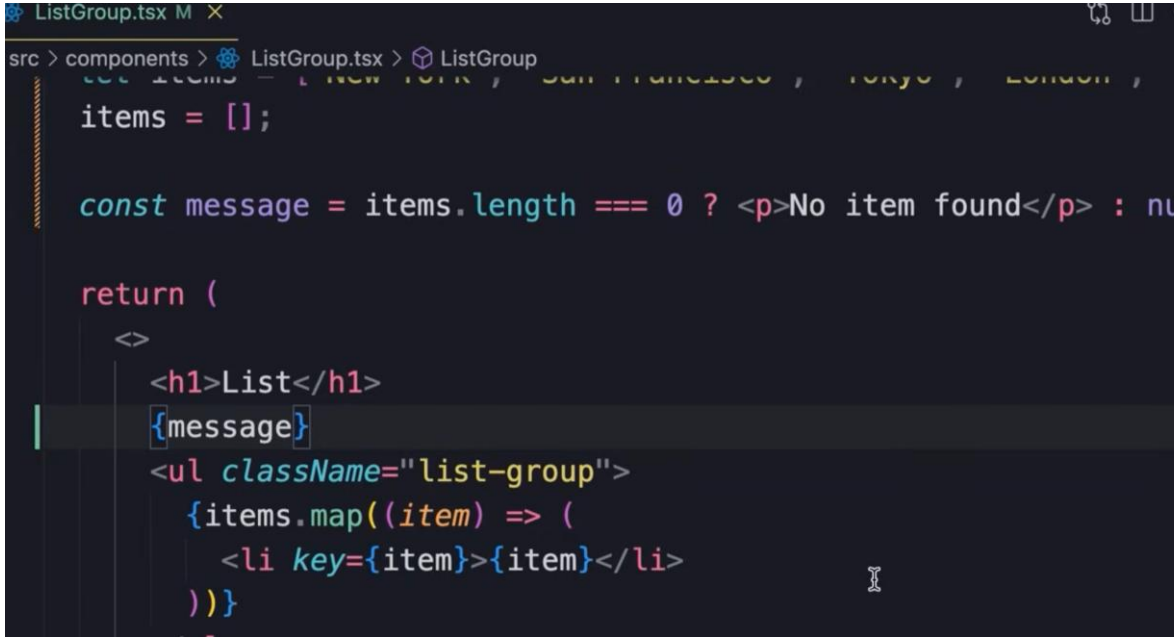
src > components > ListGroup.tsx > ListGroup > items.map() callback

```
const items = ["New York", "San Francisco", "Tokyo", "London"

return (
  <>
    <h1>List</h1>
    <ul className="list-group">
      {items.map((item) => (
        <li key={item}>{item}</li>
      ))}
    </ul>
  </>
);
}
```

Conditional Rendering

We can add a condition inside the JSX return

To avoid a crowded return, we can create constant and assign value

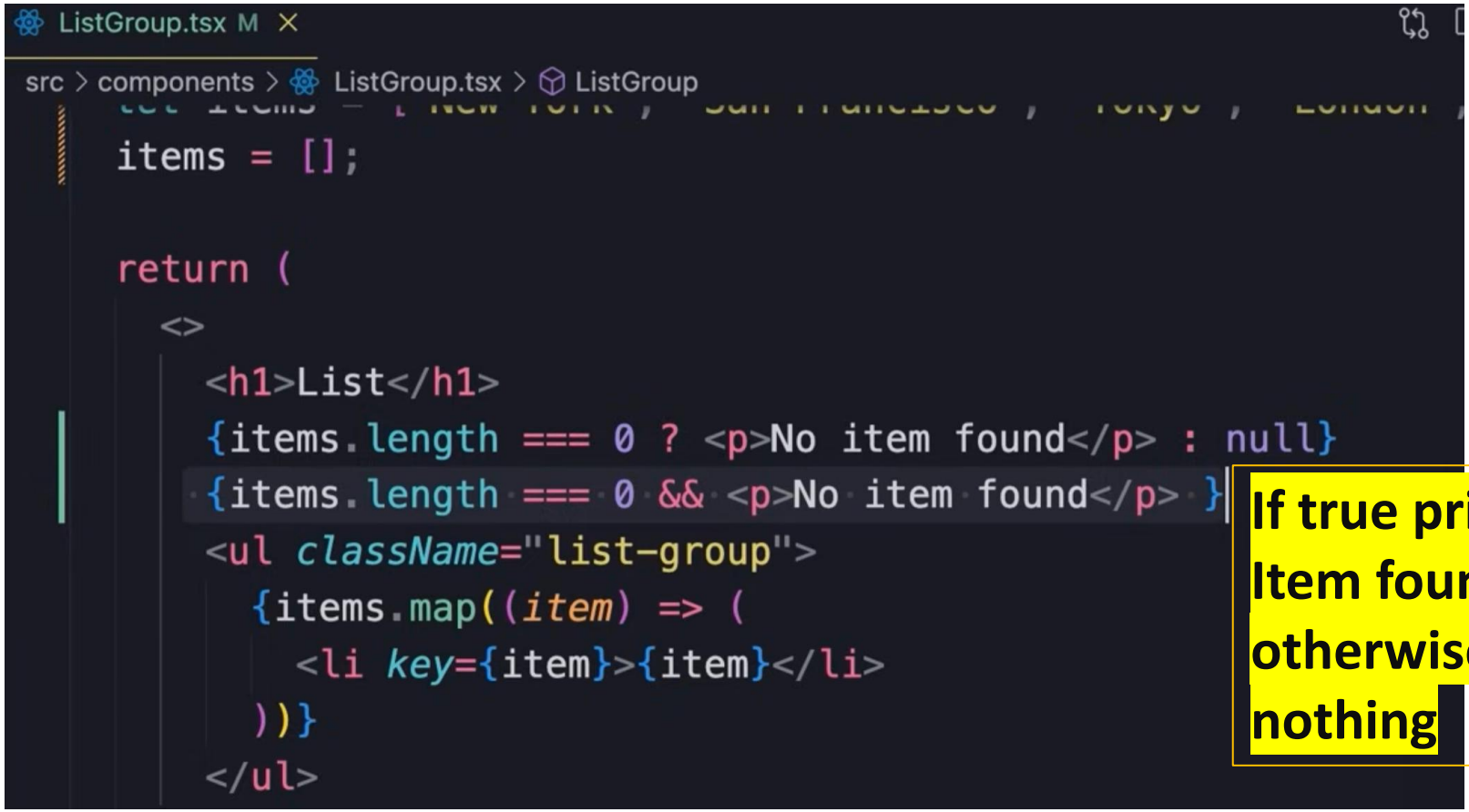


Conditional Rendering

We can even create an arrow function inside the component function, advantage here that this arrow function “getMessage” can take parameters

Conditional Rendering

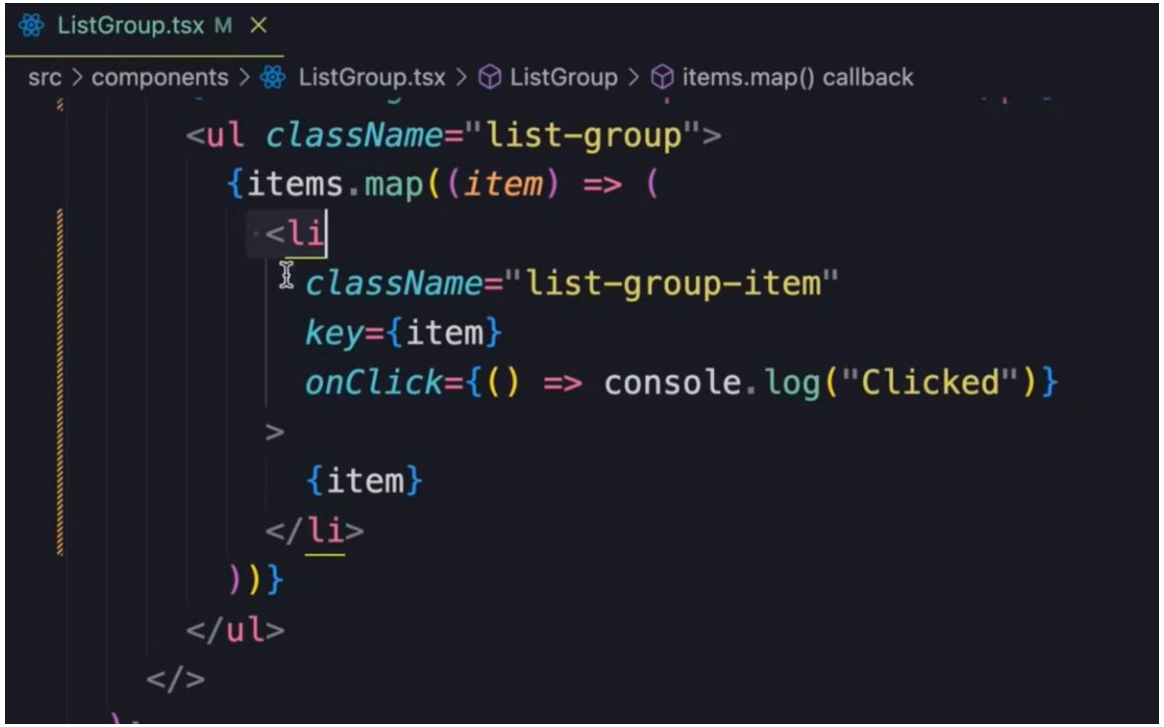
Avoid writing or Null condition, shortcut with "&&"

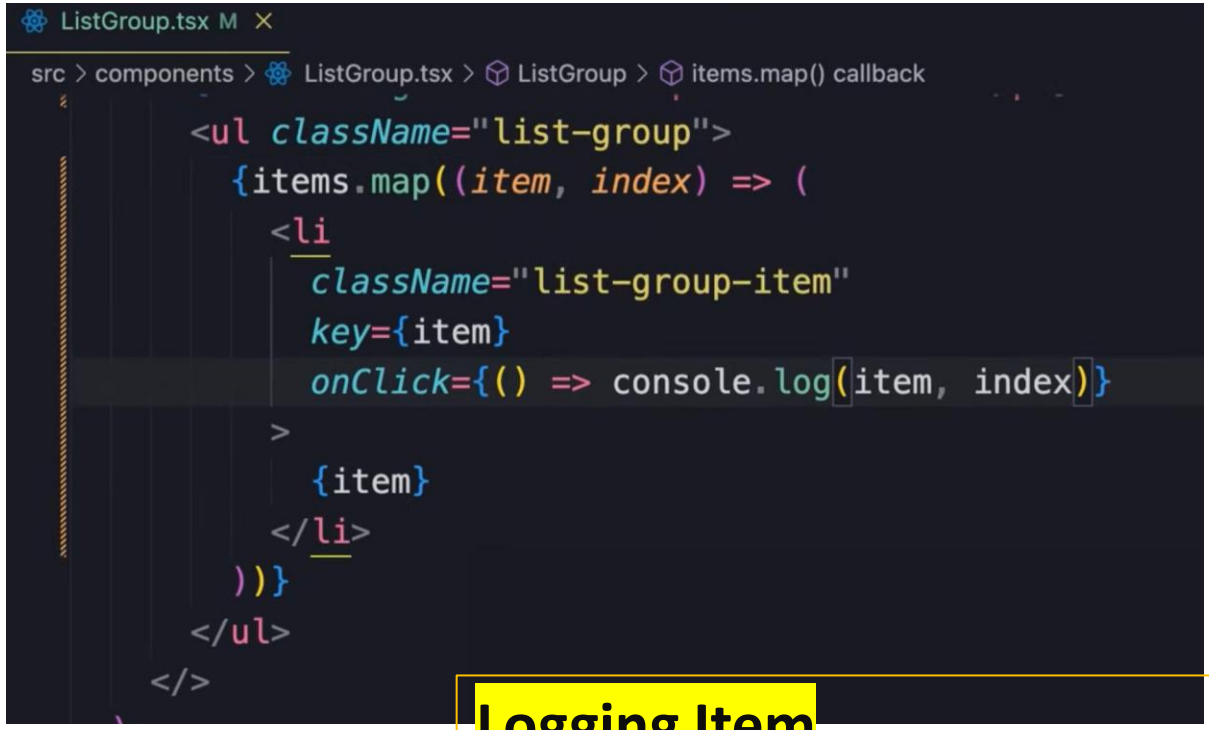


If true print "Not Item found" otherwise does nothing

Handling Events in a Component

Add the attribute “OnClick” directly and assign a function





Logging Item

Handling Events in a Component

We can even create a function:

ListGroup.tsx M

src > components > ListGroup.tsx > ListGroup

```tsx
import { MouseEvent } from "react";

function ListGroup() {
  let items = ["New York", "San Francisco", "Tokyo", "London", "
  // Event handler
  const handleClick = (event: MouseEvent) => console.log(event);

  return (
    <>
      <h1>List</h1>
```

ListGroup.tsx M

src > components > ListGroup.tsx > ListGroup > items.map() callbac

```tsx
        <ul className="list-group">
          {items.map((item, index) => (
            <li
              className="list-group-item"
              key={item}
              onClick={handleClick}
            >
              {item}
            </li>
          ))}
        </ul>
      </>
    );
```

Pass a reference

Dynamically Assigning Style Classes

```
function ListGroup() {
  let items = ["New York", "San Francisco", "Tokyo", "London", "P
  let selectedIndex = 0;

  // Event handler
  const handleClick = (event: MouseEvent) => console.log(event);

  return (
    <>
      <h1>List</h1>
      {items.length === 0 && <p>No item found</p>}
```

0 means the first item is selected

```
      <li
        className={
          selectedIndex === index
            ? "list-group-item active"
            : "list-group-item"
        }
        key={item}
        onClick={() => { selectedIndex = index;}}
      >
        {item}
```

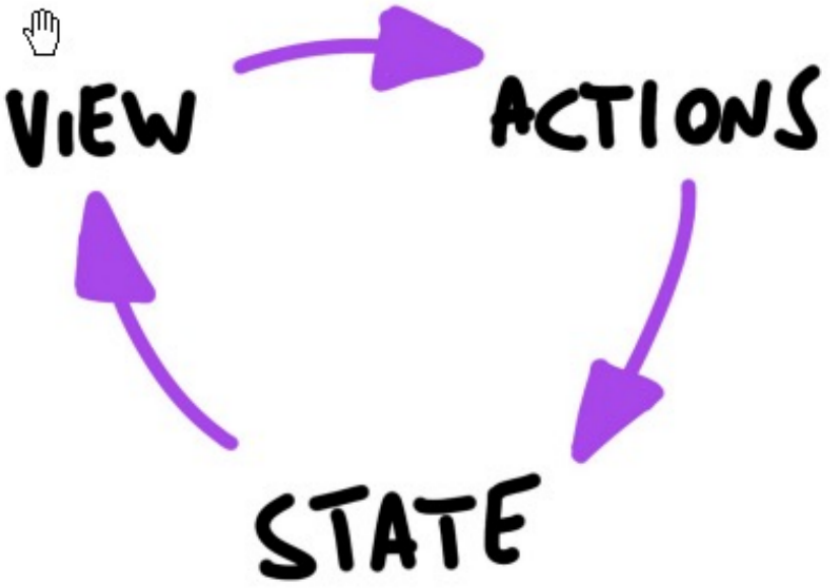
Use it in the “Condition”, but there is a problem: “selectedIndex” variable will not change . “selectedIndex” is local to the function component and React cannot notice its change like that!

React State

- **State** is a set of variables local to the component
  - Initialized with default value or by props' values
  - Can be mutated only by calling specific methods
    - Asynchronous
    - Will initiate re-rendering of the Virtual DOM
  - Current state value can be passed to children (as props)

Unidirectional Data Flow

- State is passed to the view and to child components
- Actions are triggered by the view
- Actions can update the state
- The state change is passed to the view and to child component



Unidirectional Data Flow

- A **state** is always **owned by one Component**
  - Any data that's affected by this state can only affect Components below it: its children.
- Changing state on a Component will never affect its parent, or its siblings, or any other Component in the application
  - Just its children
- For this reason, state is often **moved up** in the Component tree, so that it can be **shared** between components that need to access it.

```
<Container>
  <h1>Hello</h1>
  <p>Welcome to my site.</p>
</Container>
```

In React, **component children** refers to **whatever you place inside a component's opening and closing tags**.

Here:
- `<h1>Hello</h1>`
- `<p>Welcome to my site.</p>`

are the **children** of the `Container` component.

Using state that will change! UseState Hook

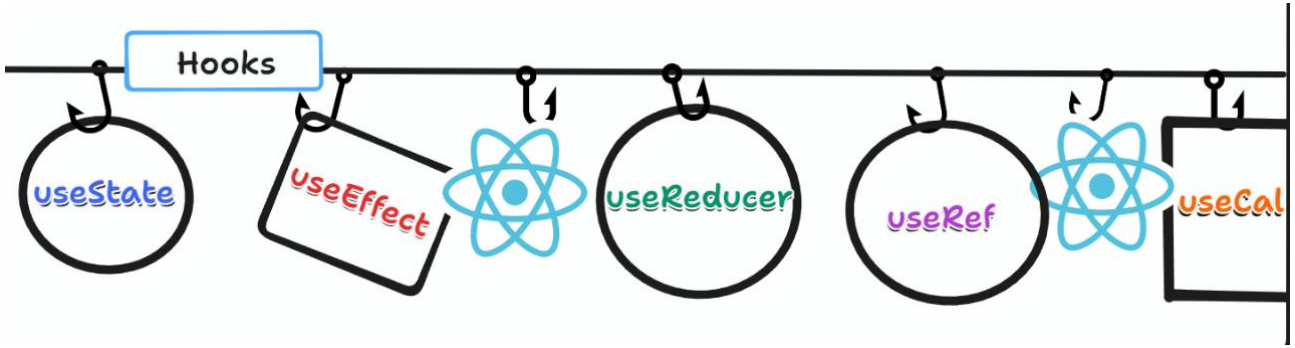
Instead of Local function variables, we use "**State Variables**" and we use the "UseState" Hook. One of many React Hooks

Hooks: Special functions that allow developers to hook into **state** and **lifecycle** of React components.

State: One or more data values associated with a React component instance.

Lifecycle: The events associated with a React component instance (create, render, destroy, etc).

In React, we think of components with states. When states of a component changes, React automatically update the DOM; we almost don't need to worry about updating the DOM manually.



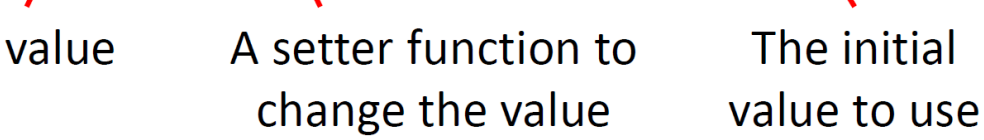
React hook: useState

Purpose:

1. Remember values internally when the component re-renders
2. Tell React to re-render the component when the value changes

Syntax:

```
const [val, setVal] = useState(100);
```



Using state that will change! UseState Hook

src > components > ListGroup.tsx > ListGroup

```tsx
import { useState } from "react";

function ListGroup() {
  let items = ["New York", "San Francisco", "Tokyo", "London", "F
  const [selectedIndex, setSelectedIndex] = useState(-1);
```

ListGroup.tsx 1, M

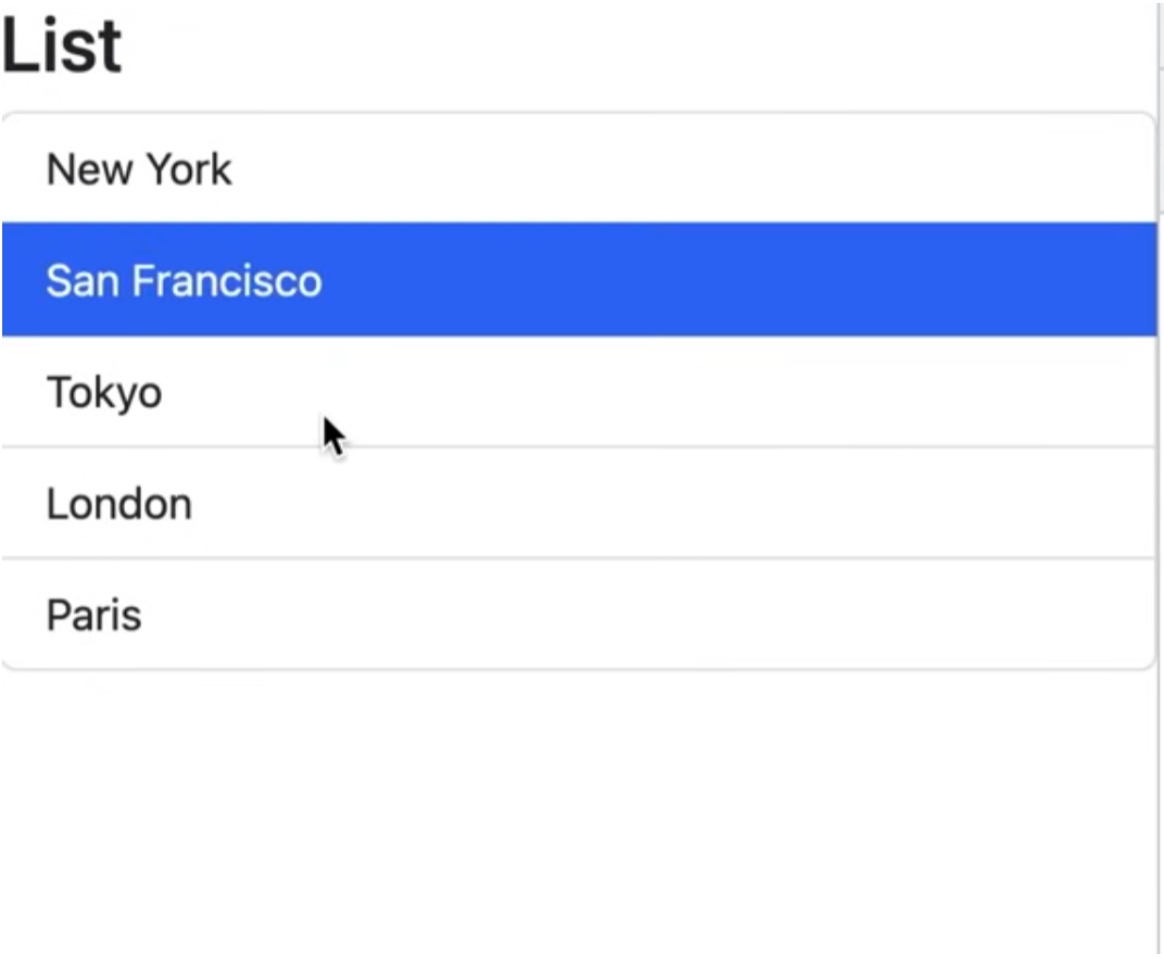
src > components > ListGroup.tsx > ListGroup > items.map() callback

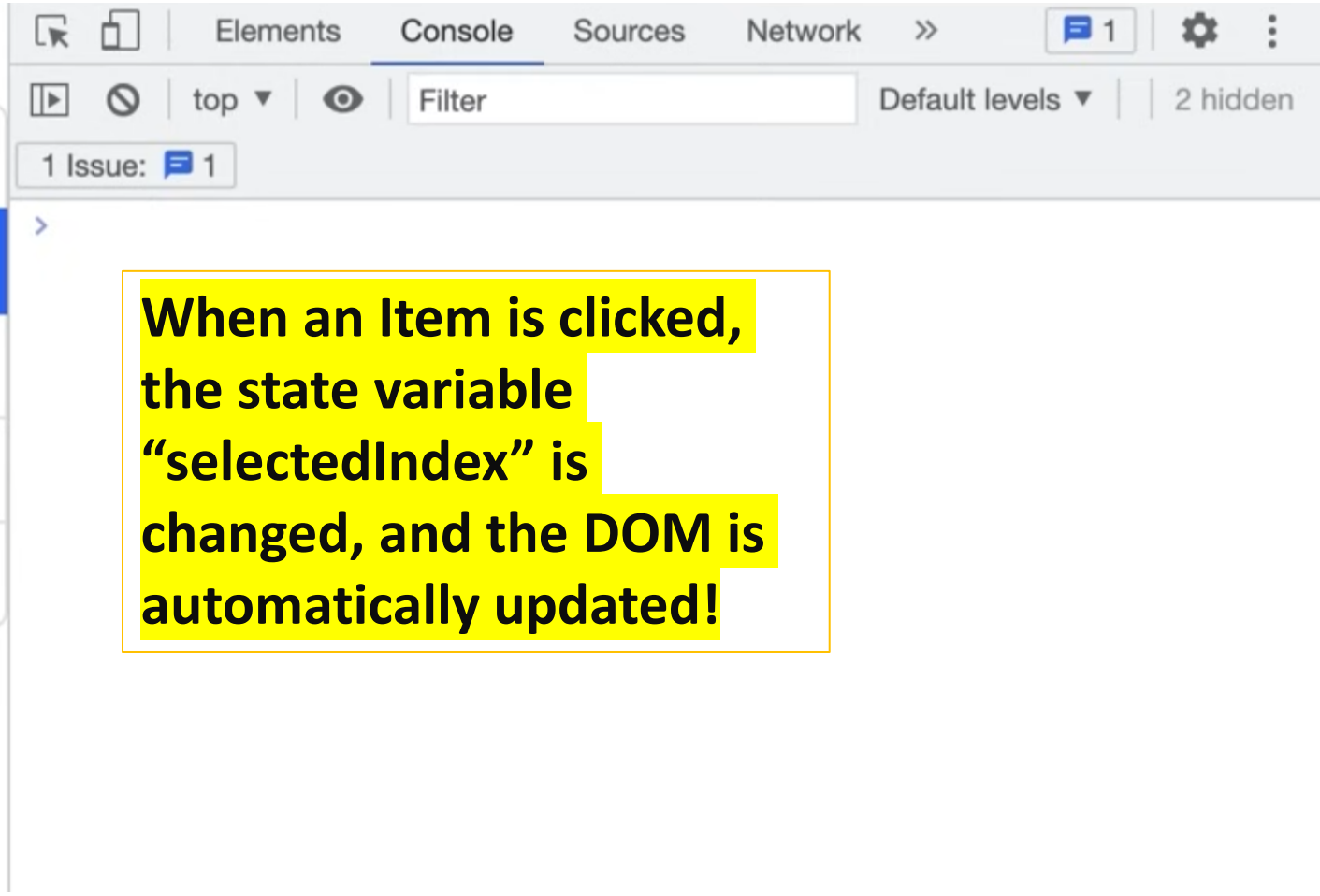
```tsx
            className={
              selectedIndex === index
                ? "list-group-item active"
                : "list-group-item"
            }
            key={item}
            onClick={() => { setSelectedIndex(index); }}
          >
            {item}
          </li>
        ))}
      </ul>
```

Create a state variable called "selectedIndex" with default "-1". And assign setSet function "setSelectedIndex" function to update that state

Using the set state function "**setSelectedIndex**"

Using state that will change! UseState Hook





When an Item is clicked, the state variable "selectedIndex" is changed, and the DOM is automatically updated!

Thank You…